

aula11

April 24, 2026

1 Tipos Algébricos

Para além das listas definidas indutivamente, podemos definir outros tipos de dados. Um exemplo é o tipo árvore binária.

1.1 Árvores binárias

Uma árvore binária é um tipo de dado que pode ser definido indutivamente como:

```
[1]: type 'a tree = Empty | Node of 'a * 'a tree * 'a tree
```

```
[1]: type 'a tree = Empty | Node of 'a * 'a tree * 'a tree
```

Um exemplo de uma árvore binária com o seguinte aspeto é:

```
  2
 / \
1   3
```

```
[2]: let t0 = Node(2,
                  Node(1,
                      Empty,
                      Empty),
                  Node(3,
                      Empty,
                      Empty))
```

```
[2]: val t0 : int tree = Node (2, Node (1, Empty, Empty), Node (3, Empty, Empty))
```

Para inserir um elemento numa árvore binária em OCaml vamos gerar a árvore binária que corresponder à inserção do elemento. Vamos fazê-lo indutivamente na estrutura da árvore. Há muitas alternativas para inserir um elemento numa árvore binária. Uma delas é a seguinte:

```
[3]: let rec insert x t =
      match t with
      | Empty -> Node(x, Empty, Empty)
      | Node(y,l,r) -> Node(x,t,Empty)
```

```
let _ = assert (insert 0 t0 = Node (0, Node (2, Node (1, Empty, Empty), Node (3, Empty, Empty)), Empty))
```

```
[3]: val insert : 'a -> 'a tree -> 'a tree = <fun>
```

```
[3]: - : unit = ()
```

1.2 Árvores binárias de pesquisa

Uma árvore binária de pesquisa é uma árvore binária com a seguinte propriedade: para cada nó, o valor do nó é maior que todos os valores da subárvore esquerda e menor que todos os valores da subárvore direita. Adicionalmente a subárvore esquerda e a subárvore direita também são árvores binárias de pesquisa.

A inserção de um elemento numa árvore binária de pesquisa de forma a manter a propriedade de árvore binária de pesquisa é feita da seguinte forma:

```
[4]: let rec insert_bst x t =  
    match t with  
    | Empty -> Node(x, Empty, Empty)  
    | Node(y,l,r) -> if x < y then Node(y,insert_bst x l,r) else Node(y,l,insert_bst x r)  
  
    let t1 = insert_bst 0 t0  
    let t2 = Node(4, t0, Empty)  
    let _ = assert(t1 = Node (2, Node (1, Node (0, Empty, Empty), Empty), Node (3, Empty, Empty)))
```

```
[4]: val insert_bst : 'a -> 'a tree -> 'a tree = <fun>
```

```
[4]: val t1 : int tree =  
    Node (2, Node (1, Node (0, Empty, Empty), Empty), Node (3, Empty, Empty))
```

```
[4]: val t2 : int tree =  
    Node (4, Node (2, Node (1, Empty, Empty), Node (3, Empty, Empty)), Empty)
```

```
[4]: - : unit = ()
```

Vamos definir uma função auxiliar `remove_min` que remove o nó com o menor valor de uma árvore binária de pesquisa. Esta função vai ser útil para remover um nó da árvore binária de pesquisa. A função `remove_min` vai devolver a árvore binária de pesquisa sem o nó com o menor valor.

```
[5]: (** [remove_min t] denotes the tree removing the minimum value from [t]
      pre: [t <> Empty] *)
let rec remove_min t =
  match t with
  | Empty -> failwith "Precondition violated!"
  | Node(x,Empty,r) -> r
  | Node(x,l,r) -> Node(x, remove_min l, r)

(** [min_tree t] denotes the smaller value in [t]
    pre: [t <> Empty] *)
let rec min_tree t =
  match t with
  | Empty -> failwith "Precondition violated!"
  | Node(x,Empty,r) -> x
  | Node(x,l,r) -> min_tree l

(** [min_tree t] denotes the larger value in [t]
    pre: [t <> Empty] *)
let rec max_tree t =
  match t with
  | Empty -> failwith "Precondition violated!"
  | Node(x,_,Empty) -> x
  | Node(_,_,r) -> max_tree r
```

```
[5]: val remove_min : 'a tree -> 'a tree = <fun>
```

```
[5]: val min_tree : 'a tree -> 'a = <fun>
```

```
[5]: val max_tree : 'a tree -> 'a = <fun>
```

Dadas estas funções auxiliares, podemos definir uma função que corresponde à prova indutiva da propriedade das árvores binárias de pesquisa. Se considerarmos que a chamada recursiva para a subárvore esquerda e para a subárvore direita corresponde à hipótese de indução (a propriedade para uma árvore mais pequena), então podemos definir o resultado para a árvore composta verificando se todos os elementos na subárvore esquerda são menores que a raiz e todos os elementos na subárvore direita são maiores que a raiz.

```
[6]: let rec is_a_bst t =
      match t with
      (* Base case *)
      | Empty -> true

      (* Base case *)
      | Node(_, Empty,Empty) -> true
```

```

(* Inductive case if the left subtree is empty *)
| Node(x,Empty,r) -> (* by h.i. *) is_a_bst r && min_tree r >= x

(* Inductive case if the right subtree is empty *)
| Node(x,l,Empty) -> (* by h.i. *) is_a_bst l && max_tree l < x

(* General Inductive case *)
| Node(x,l,r) -> (* by h.i. *) is_a_bst l && (* by h.i. *) is_a_bst r &&
↳max_tree l < x && min_tree r >= x

let _ = assert(is_a_bst t0)
let _ = assert(is_a_bst t1)
let _ = assert(is_a_bst t2)

```

```
[6]: val is_a_bst : 'a tree -> bool = <fun>
```

```
[6]: - : unit = ()
```

```
[6]: - : unit = ()
```

```
[6]: - : unit = ()
```

A remoção de um valor de uma árvore binária de pesquisa é feita procurando o valor e removendo-o. Quando um nó intermédio é removido, é necessário substituí-lo pelo valor mínimo da subárvore direita (ou pelo valor máximo da subárvore esquerda). Isto é feito chamando a função `remove_min` na subárvore direita e substituindo o nó intermédio pelo resultado. A subárvore esquerda permanece inalterada. O resultado é uma nova árvore binária de pesquisa com as mesmas propriedades de antes, mas sem o valor removido.

```

[7]: let rec remove x t =
  match t with
  | Empty -> Empty
  | Node(y,l,r) ->
    if x = y then
      Node(min_tree r, l, remove_min r)
    else
      if x < y then
        Node(y, remove x l ,r)
      else
        Node(y, l, remove x r)

let _ = assert (remove 2 t0 = Node (3, Node (1, Empty, Empty), Empty))
let _ = assert (remove 2 t2 = Node (4, Node (3, Node (1, Empty, Empty), Empty),
↳Empty))

```

```
[7]: val remove : 'a -> 'a tree -> 'a tree = <fun>
```

```
[7]: - : unit = ()
```

```
[7]: - : unit = ()
```

- binary trees
- insert in a bst
- remove in a bst (remove_min)
- tree_size
- depth
- pre_order
- in_order
- post_order
- map
- fold_in_order
- fold_pre_order
- fold_post_order

Uma alternativa à função `remove_min` acima é calcular o valor mínimo ao mesmo tempo que este é removido. A função `remove` é então definida da seguinte forma:

Outras funções sobre árvores que podemos definir são as seguintes:

```
[8]: let rec tree_size t =  
    match t with  
    | Empty -> 0  
    | Node(_, l, r) -> 1 + (tree_size l) + (tree_size r)  
  
    let rec search_bst x t =  
        match t with  
        | Empty -> false  
        | Node(y, l, r) -> x = y || (x < y && (search_bst x l)) || search_bst x r  
  
    let rec tree_depth t =  
        match t with  
        | Empty -> 0  
        | Node(_, l, r) -> 1 + max (tree_depth l) (tree_depth r)  
  
    let _ = assert (4 = tree_size t2)  
    let _ = assert (3 = tree_depth t2)
```

```
[8]: val tree_size : 'a tree -> int = <fun>
```

```
[8]: val search_bst : 'a -> 'a tree -> bool = <fun>
```

```
[8]: val tree_depth : 'a tree -> int = <fun>
```

```
[8]: - : unit = ()
```

```
[8]: - : unit = ()
```

```
[9]: let rec tree_size t =  
      match t with  
      | Empty -> 0  
      | Node(_, l, r) -> 1 + tree_size l + tree_size r  
  
      let rec tree_depth t =  
          match t with  
          | Empty -> 0  
          | Node(_, l, r) -> 1 + max (tree_depth l) (tree_depth r)  
  
      let _ = assert (4 = tree_size t2)  
      let _ = assert (3 = tree_depth t2)
```

```
[9]: val tree_size : 'a tree -> int = <fun>
```

```
[9]: val tree_depth : 'a tree -> int = <fun>
```

```
[9]: - : unit = ()
```

```
[9]: - : unit = ()
```

2 Percursos em árvores binárias

Os percursos em árvores binárias são funções que percorrem a árvore de uma forma específica. Existem três tipos de percursos em árvores binárias:

- prefixo (pre-order): visita a raiz, depois a subárvore esquerda e depois a subárvore direita
- infixo (in-order): visita a subárvore esquerda, depois a raiz e depois a subárvore direita
- posfixo (post-order): visita a subárvore esquerda, depois a subárvore direita e depois a raiz

```
[10]: let t = Node(1,Empty,Node(2,Empty,Node (3,Empty,Empty)))
      let t3 = Node(0, t, t)

      let rec pre_order t =
        match t with
        | Empty -> []
        | Node(x,l,r) ->
            x :: (pre_order l) @ (pre_order r)

      let _ = pre_order t3
      let _ = pre_order t2
```

```
[10]: val t : int tree = Node (1, Empty, Node (2, Empty, Node (3, Empty, Empty)))
```

```
[10]: val t3 : int tree =
      Node (0, Node (1, Empty, Node (2, Empty, Node (3, Empty, Empty))),
        Node (1, Empty, Node (2, Empty, Node (3, Empty, Empty))))
```

```
[10]: val pre_order : 'a tree -> 'a list = <fun>
```

```
[10]: - : int list = [0; 1; 2; 3; 1; 2; 3]
```

```
[10]: - : int list = [4; 2; 1; 3]
```

```
[11]: let rec in_order t =
      match t with
      | Empty -> []
      | Node(x,l,r) ->
          (in_order l) @ [x] @ (in_order r)

      let _ = in_order t2
```

```
[11]: val in_order : 'a tree -> 'a list = <fun>
```

```
[11]: - : int list = [1; 2; 3; 4]
```

```
[12]: let rec post_order t =
      match t with
      | Empty -> []
      | Node(x,l,r) ->
          (post_order l) @ (post_order r) @ [x]
```

```
let _ = post_order t2
```

```
[12]: val post_order : 'a tree -> 'a list = <fun>
```

```
[12]: - : int list = [1; 3; 2; 4]
```

```
[13]: let rec pre_order t =  
      match t with  
      | Empty -> []  
      | Node(x,l,r) -> x::(pre_order l)@(pre_order r)  
  
      let _ = assert([0;1;2;3;1;2;3] = pre_order t3)
```

```
[13]: val pre_order : 'a tree -> 'a list = <fun>
```

```
[13]: - : unit = ()
```

```
[14]: let rec in_order t =  
      match t with  
      | Empty -> []  
      | Node(x,l,r) -> (in_order l)@[x]@(in_order r)  
  
      let _ = assert([1;2;3;0;1;2;3] = in_order t3)
```

```
[14]: val in_order : 'a tree -> 'a list = <fun>
```

```
[14]: - : unit = ()
```

```
[15]: let ot =  
      ↪Node(4,Node(2,Node(1,Empty,Empty),Node(3,Empty,Empty)),Node(6,Node(5,Empty,Empty),Node(7,Em  
  
      let _ = assert([1; 2; 3; 4; 5; 6; 7] = in_order ot)  
      let _ = assert([4; 2; 1; 3; 6; 5; 7] = pre_order ot)
```

```
[15]: val ot : int tree =  
      Node (4, Node (2, Node (1, Empty, Empty), Node (3, Empty, Empty)),  
      Node (6, Node (5, Empty, Empty), Node (7, Empty, Empty)))
```

```
[15]: - : unit = ()
```



```
[15]: - : unit = ()
```

```
[16]: let rec post_order t =  
      match t with  
      | Empty -> []  
      | Node(x,l,r) -> (post_order l)@(post_order r)@[x]  
  
      let _ = assert( [1; 3; 2; 5; 7; 6; 4] = post_order ot)
```

```
[16]: val post_order : 'a tree -> 'a list = <fun>
```

```
[16]: - : unit = ()
```

```
[17]: let rec in_order_rev t =  
      match t with  
      | Empty -> []  
      | Node(x,l,r) -> (in_order_rev r)@[x]@(in_order_rev l)  
  
      let _ = assert( [7; 6; 5; 4; 3; 2; 1] = in_order_rev ot)
```

```
[17]: val in_order_rev : 'a tree -> 'a list = <fun>
```

```
[17]: - : unit = ()
```

3 Funções de ordem superior (Map e Fold)

```
[18]: let rec map_lst f l =  
      match l with  
      | [] -> []  
      | x::xs -> f x :: map_lst f xs  
  
      let map_opt f o =  
        match o with  
        | None -> None  
        | Some x -> Some (f x)  
  
        let _ = assert (map_opt (fun x -> x + 1) (Some 1) = Some 2)  
        let _ = assert (map_opt (fun x -> x + 1) None = None)  
  
        let _ = assert (map_lst ((+) 1) [1;2;3] = [2;3;4])
```

```
[18]: val map_lst : ('a -> 'b) -> 'a list -> 'b list = <fun>
```

```
[18]: val map_opt : ('a -> 'b) -> 'a option -> 'b option = <fun>
```

```
[18]: - : unit = ()
```

```
[18]: - : unit = ()
```

```
[18]: - : unit = ()
```

```
[19]: let rec map f t =  
      match t with  
      | Empty -> Empty  
      | Node(x,l,r) -> Node(f x, map f l, map f r)  
  
      let _ = ot  
      let _ = map (fun x -> x+1) ot
```

```
[19]: val map : ('a -> 'b) -> 'a tree -> 'b tree = <fun>
```

```
[19]: - : int tree =  
      Node (4, Node (2, Node (1, Empty, Empty), Node (3, Empty, Empty)),  
            Node (6, Node (5, Empty, Empty), Node (7, Empty, Empty)))
```

```
[19]: - : int tree =  
      Node (5, Node (3, Node (2, Empty, Empty), Node (4, Empty, Empty)),  
            Node (7, Node (6, Empty, Empty), Node (8, Empty, Empty)))
```

```
[20]: let rec map f t =  
      match t with  
      | Empty -> Empty  
      | Node(x,l,r) -> Node(f x, map f l, map f r)  
  
      let _ = map string_of_int ot  
      let _ = map ((+)1) ot
```

```
[20]: val map : ('a -> 'b) -> 'a tree -> 'b tree = <fun>
```

```
[20]: - : string tree =  
Node ("4", Node ("2", Node ("1", Empty, Empty), Node ("3", Empty, Empty)),  
Node ("6", Node ("5", Empty, Empty), Node ("7", Empty, Empty)))
```

```
[20]: - : int tree =  
Node (5, Node (3, Node (2, Empty, Empty), Node (4, Empty, Empty)),  
Node (7, Node (6, Empty, Empty), Node (8, Empty, Empty)))
```

```
[21]: let map_opt f o =  
      match o with  
      | None -> None  
      | Some v -> Some (f v)  
  
let _ = map_opt string_of_int (Some 1)  
let _ = map_opt string_of_int None  
let _ = map_opt ((+)1) (Some 1)  
let _ = map_opt ((+)1) None
```

```
[21]: val map_opt : ('a -> 'b) -> 'a option -> 'b option = <fun>
```

```
[21]: - : string option = Some "1"
```

```
[21]: - : string option = None
```

```
[21]: - : int option = Some 2
```

```
[21]: - : int option = None
```

```
[22]: let rec fold_in_order f acc t =  
      match t with  
      | Empty -> acc  
      | Node(x, l, r) ->  
          let acc2 = fold_in_order f acc l in  
          let acc3 = f acc2 x in  
          fold_in_order f acc3 r  
  
let _ = List.fold_left (fun acc x -> acc+x) 0 [1;2;3]  
  
let _ = fold_in_order (+) 0 ot  
  
let _ = fold_in_order (fun acc x -> acc@[x]) [] ot
```

```
let _ = List.rev (fold_in_order (fun acc x -> x::acc) [] ot)
```

```
[22]: val fold_in_order : ('a -> 'b -> 'a) -> 'a -> 'b tree -> 'a = <fun>
```

```
[22]: - : int = 6
```

```
[22]: - : int = 28
```

```
[22]: - : int list = [1; 2; 3; 4; 5; 6; 7]
```

```
[22]: - : int list = [1; 2; 3; 4; 5; 6; 7]
```

```
[23]: let rec fold_left f acc l =  
      match l with  
      | [] -> acc  
      | x::xs -> fold_left f (f acc x) xs  
  
      let _ = fold_left (+) 0 (pre_order ot)  
  
      let rec fold_in_order f acc t =  
        match t with  
        | Empty -> acc  
        | Node(x,l,r) -> let acc_l = fold_in_order f acc l in let acc_root = f acc_l x  
        ↪ x in fold_in_order f acc_root r  
  
      let _ = fold_in_order (+) 0 ot  
      let _ = fold_in_order (fun acc x -> acc@[x]) [] ot  
      let filter p t = fold_in_order (fun acc x -> if p x then acc@[x] else acc) [] ot  
  
      let _ = filter (fun x -> x mod 2 = 0) ot
```

```
[23]: val fold_left : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a = <fun>
```

```
[23]: - : int = 28
```

```
[23]: val fold_in_order : ('a -> 'b -> 'a) -> 'a -> 'b tree -> 'a = <fun>
```

```
[23]: - : int = 28
```

```
[23]: - : int list = [1; 2; 3; 4; 5; 6; 7]
```

```
[23]: val filter : (int -> bool) -> 'a -> int list = <fun>
```

```
[23]: - : int list = [2; 4; 6]
```

```
[24]: let rec fold_pre_order f acc t =  
    match t with  
    | Empty -> acc  
    | Node(x,l,r) -> let acc_root = f acc x in let acc_l = fold_pre_order f  
    ↪acc_root l in fold_pre_order f acc_l r  
  
let _ = fold_pre_order (fun acc x -> acc@[x]) [] ()
```

```
[24]: val fold_pre_order : ('a -> 'b -> 'a) -> 'a -> 'b tree -> 'a = <fun>
```

```
[24]: - : int list = [4; 2; 1; 3; 6; 5; 7]
```